

Async Chunking for Delayed VLA Control: Toy Benchmark Note and Next Steps

Working Note for `async_chunking_compare`

April 19, 2026

Abstract

This note summarizes a lightweight point-mass experiment for comparing delayed chunked-control strategies inspired by recent VLA deployment ideas. The current artifact is useful for intuition, but it is not yet a finished benchmark. The main value of the toy is to separate several physical intuitions: synchronous replanning wastes wall-clock time, naive asynchronous replanning suffers stale-state misalignment, train-time prefix conditioning can partially internalize delay-awareness, and future-state alignment can improve handoff quality when the planner reasons about the execution-time state rather than the stale one.

1 Executive Summary

The current implementation compares six methods:

1. **Synchronous**: wait for the next chunk before continuing.
2. **Naive Async**: overlap planning and execution, but plan from stale state.
3. **RTC Hard Stitch**: a deliberately crude negative control that freezes the committed prefix without actual inpainting guidance.
4. **Train-Time Prefix**: a toy learned continuation model that predicts a constant postfix action from stale state plus committed prefix.
5. **VLASH**: roll the committed prefix forward and plan from the estimated future execution state.
6. **VLASH + Reflex**: VLASH plus a small per-step residual controller.

The main result is narrow but useful:

- In the static-goal setting, **VLASH** is the strongest method on settle time and final error.
- The toy **Train-Time Prefix** baseline closes part of the gap without explicit future rollout, but it is slower and less reliable than **VLASH**.
- **RTC Hard Stitch** fails badly, but that should be read only as evidence that hard prefix freezing is insufficient. It is not a faithful RTC baseline.
- In the moving-goal setting, the ranking is mixed. **VLASH** remains competitive, but the current metrics do not show clean dominance over all baselines.

2 Experimental Setup

The simulator is a one-dimensional point mass with position x , velocity v , control step $\Delta t = 0.02$ s, chunk horizon $H = 18$, and inference delay $d = 8$ by default. The environment injects Gaussian velocity disturbances with standard deviation 0.035 per step.

Two scenarios are currently reported:

- **Static goal:** fixed target at $x = 10.0$.
- **Moving goal:** sinusoidal target motion with amplitude 1.2 and period 2.6 s.

The static-goal evaluation emphasizes settling and endpoint accuracy. The moving-goal evaluation emphasizes tracking quality, specifically the fraction of time inside the tolerance band and the tail mean absolute tracking error.

Important limitation. This is structurally close to an oracle-versus-student comparison. VLASH uses an exact rollout of the same nominal dynamics used by the environment, and **Train-Time Prefix** is trained on labels derived from the same analytic planner. That makes the current result much narrower than a full method benchmark.

3 Static-Goal Results

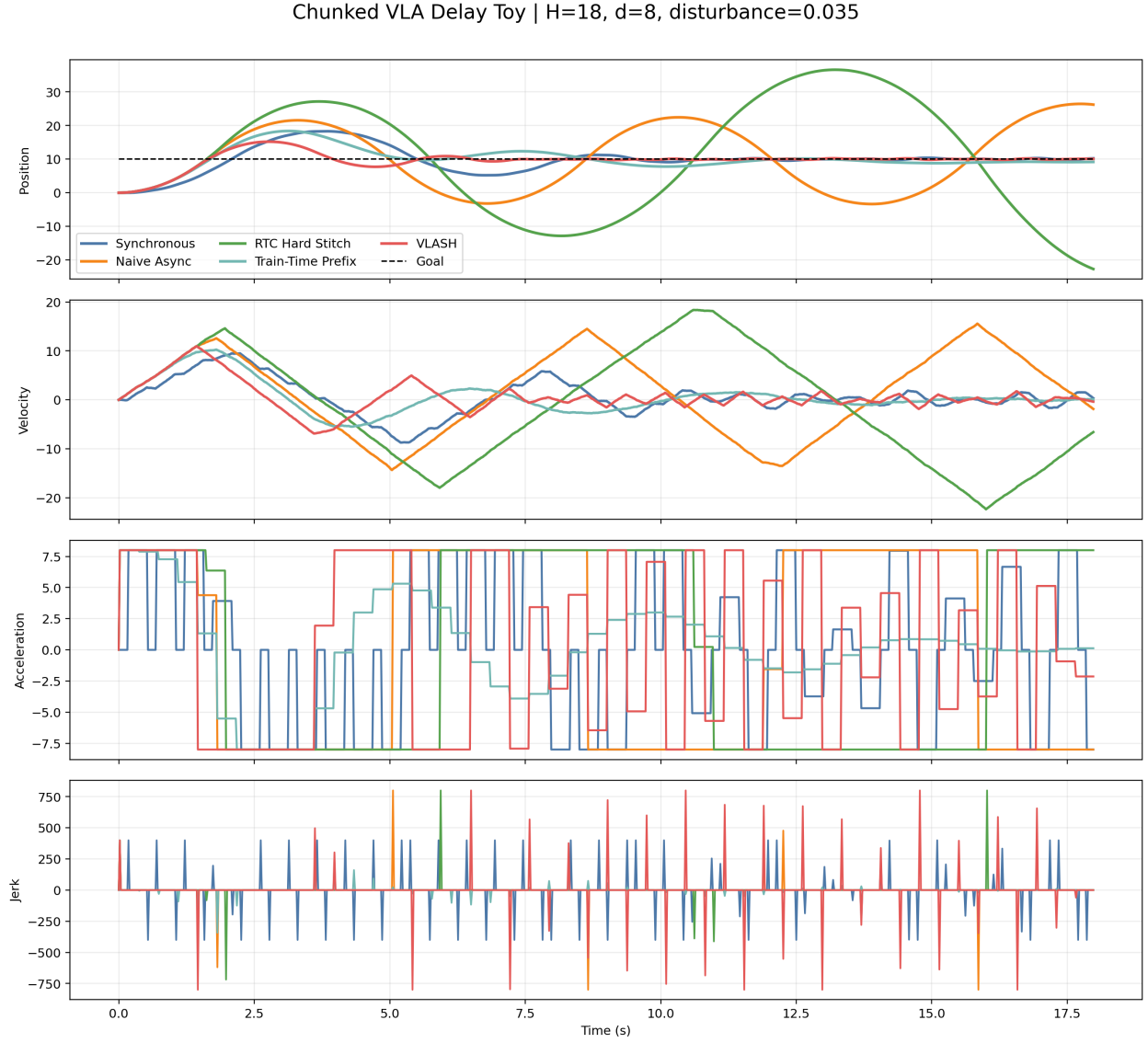


Figure 1: Single-run static-goal comparison. The main qualitative pattern is visible: synchronous replanning is stable but slow, naive async oscillates, hard stitching diverges, train-time prefix conditioning dampens the stale-state pathology, and VLASH reaches the target fastest.

Static Goal Monte Carlo Comparison

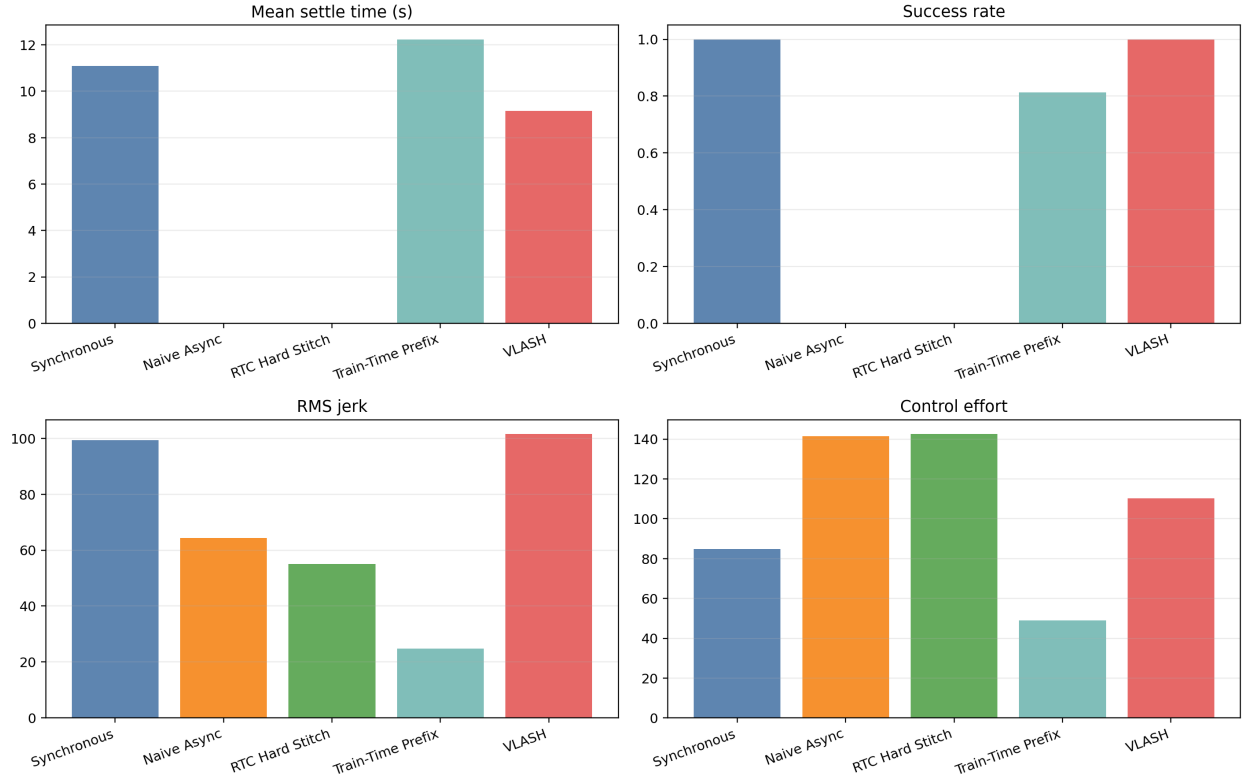


Figure 2: Monte Carlo aggregates for the static-goal setting.

Method	Success (%)	Mean settle (s)	Final error	RMS jerk
Synchronous	100.0	11.09	0.164	99.4
Naive Async	0.0	—	6.469	64.3
RTC Hard Stitch	0.0	—	22.022	55.0
Train-Time Prefix	81.2	12.24	0.649	24.8
VLASH	100.0	9.15	0.054	101.5
VLASH + Reflex	100.0	8.79	0.056	104.9

Table 1: Static-goal metrics exported by the current script.

The static-goal story is reasonably clear. **Synchronous** reaches the target reliably but pays a substantial wall-clock penalty because it pauses while replanning. **Naive Async** uses compute more efficiently, but its commands are planned for the wrong physical state and it never settles reliably. The toy **Train-Time Prefix** baseline partially repairs this by inferring a continuation from the committed prefix. **VLASH** still wins because it conditions directly on the execution-time state rather than having to infer that state from stale observations alone.

4 Moving-Goal Results

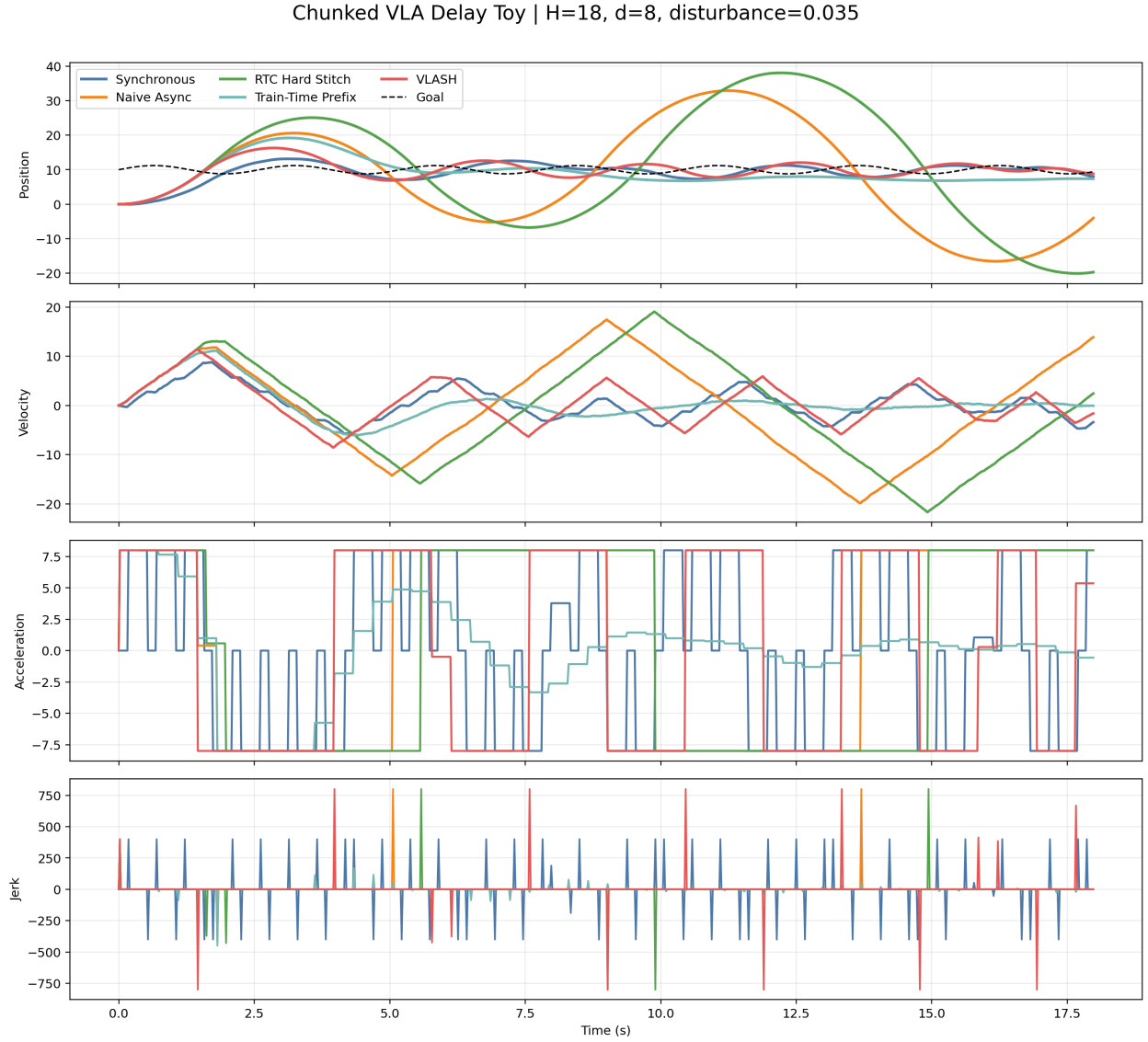


Figure 3: Single-run moving-goal comparison. The moving target increases the penalty for stale-state planning because the reference itself shifts during the delay window.

Moving Goal Monte Carlo Comparison

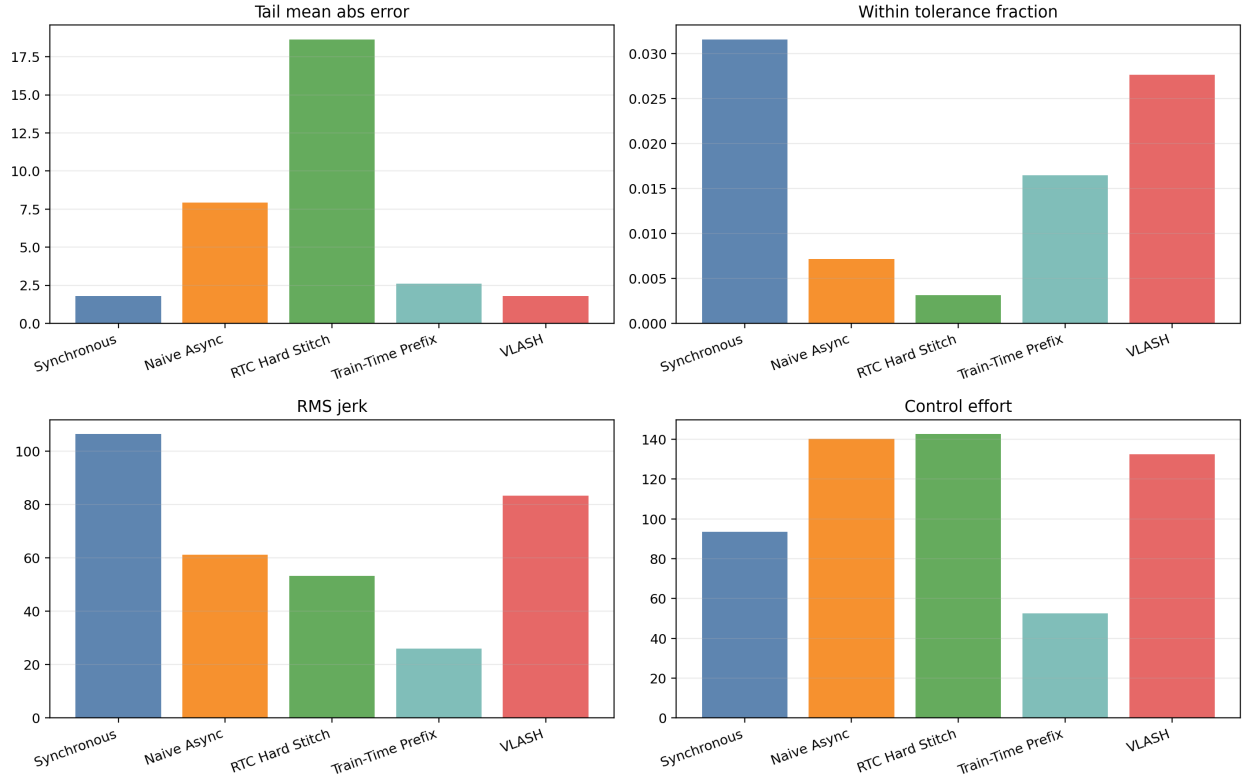


Figure 4: Monte Carlo aggregates for the moving-goal setting.

Method	Within tol. (%)	Tail MAE	Final error	RMS jerk
Synchronous	3.2	1.781	1.617	106.4
Naive Async	0.7	7.925	8.059	61.2
RTC Hard Stitch	0.3	18.645	29.850	53.2
Train-Time Prefix	1.6	2.602	1.065	26.0
VLASH	2.8	1.798	1.340	83.3
VLASH + Reflex	2.7	1.672	1.235	81.0

Table 2: Moving-goal metrics exported by the current script. Tail MAE denotes tail mean absolute error.

This scenario is more ambiguous. **Naive Async** and **RTC Hard Stitch** remain poor. **Train-Time Prefix** dramatically reduces jerk and control effort relative to the stronger planners, but it still lags the moving reference. **VLASH** and **VLASH + Reflex** stay competitive on tracking error, yet the current metrics do not justify a blanket claim of dominance over every baseline. The right reading is that future-state alignment remains promising, while the present moving-goal setup still needs better uncertainty-aware evaluation.

5 Delay Sweep

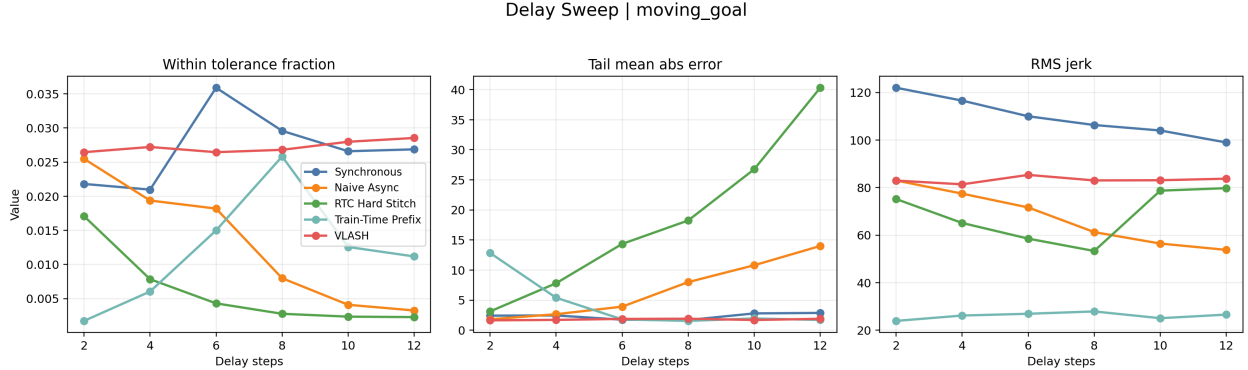


Figure 5: Moving-goal delay sweep. The current plot is most useful as an intuition aid rather than a definitive benchmark summary.

The delay sweep reinforces three narrow conclusions:

- small delays are more forgiving for all methods,
- naive async degrades quickly as delay grows,
- the hard-stitch negative control becomes unstable quickly, which supports the claim that continuity constraints need more than prefix freezing alone.

6 What This Toy Does and Does Not Establish

The current artifact does establish:

- delayed chunked control can fail even in a simple system if planning is stale,
- train-time prefix conditioning can recover part of the lost continuity,
- future-state alignment is a strong design direction in this toy.

It does *not* establish:

- that VLASH is generally best for real VLA deployment,
- that RTC is weak relative to VLASH,
- that the present moving-goal result is robust enough for strong comparative claims,
- or that the current benchmark structure is broad enough to stand in for manipulation or embodied control.

7 Next Critical Steps

The two benchmark-audit subagents converged on the same main point: the highest-value next work is not another toy variant, but turning the current comparison into a credible benchmark.

7.1 1. Fix the Baseline Set

Before extending the narrative, the comparison needs stronger baselines:

1. either add a faithful RTC-style baseline with actual inpainting-like continuity repair or remove RTC-comparison language,
2. add a history/recurrent baseline with access to stale observations and committed prefix,
3. add a classical latency-compensation controller such as MPC or Smith-predictor replanning,
4. and report reflex variants consistently if reflex control remains part of the story.

7.2 2. Break the Oracle Assumption

The current setup is too close to “future-state oracle beats a student of the same oracle.” To fix that:

1. train on one distribution and evaluate on shifted delays, horizons, disturbance magnitudes, and goal dynamics,
2. separate the teacher used for labels from the controller evaluated at test time,
3. and inject model mismatch into future-state prediction so **VLASH** is not scored with exact nominal rollout only.

7.3 3. Upgrade the Timing Model

Real async control does not have a perfectly deterministic compute delay. The benchmark should:

1. sample inference latency from a distribution,
2. include deadline misses and queueing effects,
3. and define what happens when the next chunk is late.

7.4 4. Improve Reporting Quality

The current report uses only means. The next pass should:

1. export per-trial results,
2. add confidence intervals or bootstrap intervals,
3. use paired-seed comparisons,
4. and report additional metrics such as overshoot, timeout rate, tracking lag, and safety-style violations.

7.5 5. Expand Scenario Coverage

The task family is still too narrow. The next benchmark version should include:

1. static setpoint, moving target, step changes, and adversarial disturbances,
2. sweeps over both delay and horizon,
3. and at least one higher-dimensional constrained system.

8 Current Conclusion

The present artifact is a useful toy note for reasoning about delayed chunked control, but not yet a credible benchmark. The strongest next move is to tighten the claims, add non-strawman baselines, break the oracle coupling between planner and evaluator, and rerun the study with uncertainty-aware reporting.